

10. 비전 트랜스포머

ViT

[합성곱 모델과 ViT 모델 비교](#)

[ViT의 귀납적 편향 \(Inductive Bias\)](#)

[ViT 모델](#)

[패치 임베딩 \(Patch Embedding\)](#)

[인코더 계층](#)

[모델 실습](#)

Swin Transformer

[ViT와 Swin Transformer 차이](#)

[스윈 트랜스포머 모델 구조](#)

[패치 파티션 / 패티 병합](#)

[스윈 트랜스포머 블록](#)

[모델 실습](#)

CvT(Convolutional Vision Transformer)

[합성곱 토큰 임베딩](#)

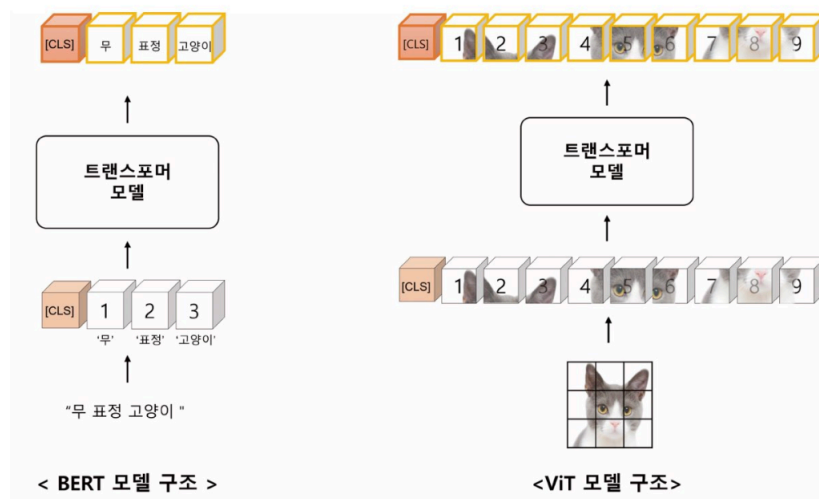
[어텐션에 대한 합성곱 임베딩](#)

[모델 실습](#)

ViT

ViT는 트랜스포머 구조 자체를 CV 분야에 적용한 첫 번째 연구다.

- BERT 모델과 ViT 모델의 차이



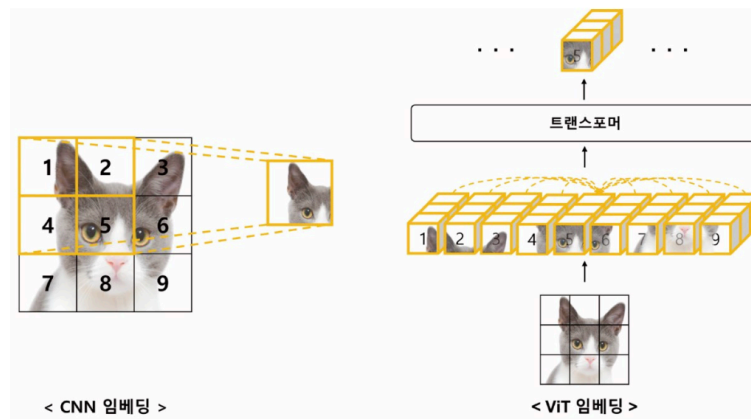
둘 다 트랜스포머 모델의 구조를 그대로 사용하지만, 입력 데이터를 만드는 과정이 서로 다르다.

- BERT : 문장보다 작은 단위의 토큰들이 순차적으로 입력됨
- ViT : 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력됨

따라서 ViT 모델에 사용되는 입력 이미지 패치는 왼쪽에서 오른쪽, 위에서 아래로 표현된 sequential 배열을 가정한다.

합성곱 모델과 ViT 모델 비교

합성곱 신경망과 트랜스포머는 이미지 특징을 잘 표현하는 임베딩을 만들고자 하는 목적은 같지만, 과정에 큰 차이가 있다.



	CNN	ViT
처리 단위	픽셀 단위로 이미지 처리	패치 단위로 이미지 처리
학습 범위	이미지 패치 중 일부만 선택해 학습	모든 이미지 패치가 학습에 관여
관계 모델링	좁은 수용 영역(Receptive Field)을 가져 전체 정보 표현에 많은 계층 필요	self-attention으로 모든 패치 간 상호 영향을 한 번에 고려 → 단일 ViT layer로 전체 표현 가능
위치 정보	공간적 절대 위치 정보 고려	패치 간 상대적 위치만 고려 → 이미지 변환에 취약할 수 있음
이미지 크기	다양한 입력 크기 처리 가능	입력 이미지 크기 고정 → 다른 크기의 이미지는 전처리 필요
모델 효율	전체 이미지 표현에 수많은 계층 필요	더 작은 모델로도 높은 성능 달성 가능

ViT의 귀납적 편향 (Inductive Bias)

귀납적 편향이란?

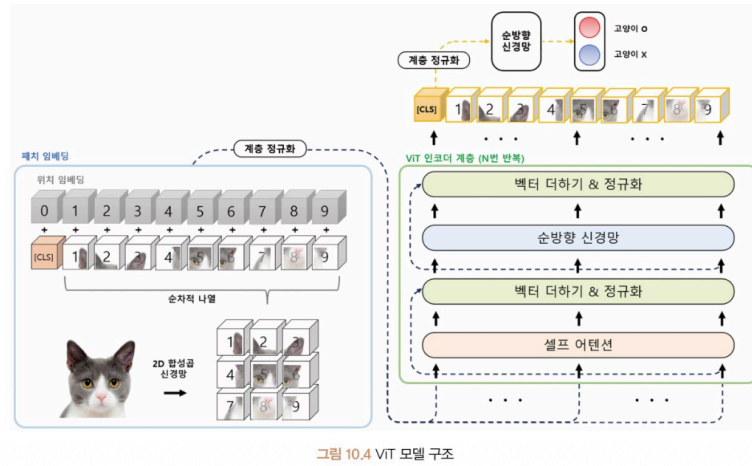
일반화 성능 향상을 위한 모델의 가정(Assumption). 가정이 맞으면 적은 데이터로도 잘 작동하지만, 너무 강하면 다른 유형의 데이터/관계 표현이 어려워짐.

모델	편향	특징
CNN	지역적(Local) 편향	공간적 관계에 강함 → 이미지에서 강점
RNN	Sequential 편향	시간적 관계에 강함 → 시계열에서 강점
ViT	거의 없음	Q/K/V로 다양한 관계를 데이터에서 직접 학습

편향이 강할수록 특정 도메인에서 잘 되지만, 다른 관계를 표현하기 어려워짐. ViT는 편향이 거의 없어서 대용량 데이터가 있을 때 이미지 특징 관계를 유연하게 학습 가능.

ViT 모델

패치 임베딩(Patch Embedding)과 인코더(Encoder) 계층으로 구성됨.

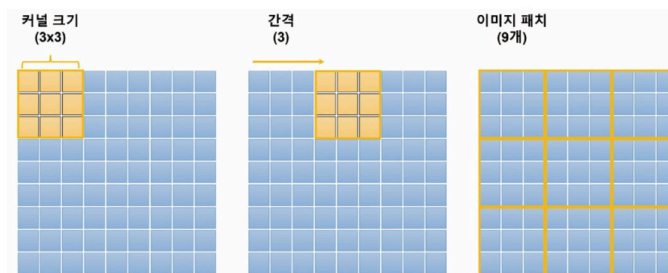


- 패치 임베딩 : 입력 이미지를 트랜스포머 구조에 맞게 일정한 크기의 패치로 나눈 다음 각 패치를 벡터 형태로 변환
- 인코더 : 각 패치와의 관계 학습

패치 임베딩과 인코더 계층을 통해 이미지의 특징을 추출하고 분류/회귀와 같은 작업에 맞는 출력값으로 변환해 사용함

패치 임베딩 (Patch Embedding)

입력 이미지를 작은 패치로 분할해 sequential 배열로 만드는 과정



- 처리 순서
 1. 이미지 resize : 입력 이미지를 정방형 크기로 조정 (e.g. 640×480 → 224×224)
 2. 패치 분할 : CNN 계층을 활용해 전체 이미지를 패치 크기로 분할
 3. [CLS] 토큰 추가 : 배열 가장 왼쪽에 분류 토큰 삽입 (전체 이미지를 대표하는 벡터)
 4. 위치 임베딩 추가 : 패치의 위치 정보를 임베딩 벡터로 변환해 패치 벡터에 더함
 5. 계층 정규화 : 최종 패치 임베딩 완성

- patch 수 계산

$$\text{patch size} = \frac{\text{image size} - \text{kernel size}}{\text{stride}} + 1$$

- e.g.

조건	패치 수
이미지 9x9, 커널 3x3, stride 3	$3 \times 3 = 9\text{개}$

조건	패치 수
이미지 9x9, 커널 3x3, stride 1	$7 \times 7 = 49$ 개

→ stride 작을수록 패치가 많아짐 (겹침 발생)

- 핵심 hyperparameter
 - Kernel Size : 패치 하나의 크기
 - Stride : 패치가 이동하는 폭
- 패치 임베딩의 장점 : 이미지를 패치 단위로 나눠 처리하면 GPU 메모리 한계를 극복하고 더 큰 이미지도 처리 가능.

인코더 계층

- 처리 흐름
 1. 패치 벡터들을 인코더 계층에 입력
 2. N개의 인코더 레이어를 반복 적용 → Q/K/V 임베딩 관계 학습 (Multi-Head Attention, Transformer와 동일)
 3. 마지막 레이어에서 [CLS] 토큰의 특징 벡터만 추출 → 전체 이미지를 대표하는 벡터로 간주
 4. 해당 벡터를 순방향 신경망(FC layer)에 연결 → 분류 문제 해결
- 핵심 포인트
 - sequential output(모든 패치 출력) 전부를 쓰는 것이 아니라, [CLS] 토큰 벡터 하나만 분류에 사용
 - 이 벡터가 이미지 분류, 검색 등 다양한 downstream task에 활용됨

모델 실습

- FashionMNIST 데이터 불러오기 균등 샘플링

```
from itertools import chain
from collections import defaultdict
from torch.utils.data import Subset
from torchvision import datasets

def subset_sampler(dataset, classes, max_len):
    target_idx = defaultdict(list)
    for idx, label in enumerate(dataset.train_labels):
        target_idx[int(label)].append(idx)

    indices = list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(dataset, indices)

train_dataset = datasets.FashionMNIST(root="./datasets", download=True,
```

```

train=True)
test_dataset = datasets.FashionMNIST(root="./datasets", download=True,
train=False)

classes = train_dataset.classes
class_to_idx = train_dataset.class_to_idx

print(classes)
print(class_to_idx)

subset_train_dataset = subset_sampler(
    dataset=train_dataset, classes=train_dataset.classes, max_len=1000
)
subset_test_dataset = subset_sampler(
    dataset=test_dataset, classes=test_dataset.classes, max_len=100
)

print(f"Training Data Size : {len(subset_train_dataset)}")
print(f"Testing Data Size : {len(subset_test_dataset)}")
print(train_dataset[0])

```

▼ 실행 결과

```

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-images-idx3-ubyte.gz to ../datasets/FashionMNIST/raw/train-images-idx3-ubyte.gz
100%|██████████| 26.4M/26.4M [00:23<00:00, 1.11MB/s]
Extracting ../datasets/FashionMNIST/raw/train-images-idx3-ubyte.gz to ../datasets/FashionMNIST/raw

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/train-labels-idx1-ubyte.gz to ../datasets/FashionMNIST/raw/train-labels-idx1-ubyte.gz
100%|██████████| 29.5k/29.5k [00:00<00:00, 103kB/s]
Extracting ../datasets/FashionMNIST/raw/train-labels-idx1-ubyte.gz to ../datasets/FashionMNIST/raw

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-images-idx3-ubyte.gz to ../datasets/FashionMNIST/raw/t10k-images-idx3-ubyte.gz
100%|██████████| 4.42M/4.42M [00:06<00:00, 644kB/s]
Extracting ../datasets/FashionMNIST/raw/t10k-images-idx3-ubyte.gz to ../datasets/FashionMNIST/raw

Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz
Downloading http://fashion-mnist.s3-website.eu-central-1.amazonaws.com/t10k-labels-idx1-ubyte.gz to ../datasets/FashionMNIST/raw/t10k-labels-idx1-ubyte.gz
100%|██████████| 5.15k/5.15k [00:00<00:00, 7.24MB/s]
Extracting ../datasets/FashionMNIST/raw/t10k-labels-idx1-ubyte.gz to ../datasets/FashionMNIST/raw

['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
{'T-shirt/top': 0, 'Trouser': 1, 'Pullover': 2, 'Dress': 3, 'Coat': 4, 'Sandal': 5, 'Shirt': 6, 'Sneaker': 7, 'Bag': 8, 'Ankle boot': 9}
Training Data Size : 1
Testing Data Size : 1
(<PIL.Image.Image image mode=L size=28x28 at 0x1043096f8>, 9)

/opt/homebrew/Caskroom/miniforge/base/envs/dl-euron/lib/python3.10/site-packages/torchvision/datasets/mnist.py:66: UserWarning: train_labels has been renamed target
warnings.warn("train_labels has been renamed targets")

```

- Hugging Face의 이미지 프로세서 클래스(AutoImageProcessor)를 사용해 전처리
모델 학습을 위해서는 PIL.Image 형식이 아닌 Tensor 형식으로 변환해야 하므로 전처리를 진행

```

import torch
from torchvision import transforms
from transformers import AutoImageProcessor

image_processor = AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="google/vit-base-patch16-224-in21k"
)

```

```

transform=transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Resize( # 학습에 사용하려는 이미지를 ViT 모델과 맞추기 위함
            size=(
                image_processor.size["height"],
                image_processor.size["width"]
            )
        ),
        transforms.Lambda( # 단일 채널 복제해 다중 채널 이미지로 변환
            lambda x: torch.cat([x,x,x], 0)
        ),
        transforms.Normalize( #정규화
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
    ]
)

print(f"size: {image_processor.size}")
print(f"mean: {image_processor.image_mean}")
print(f"std: {image_processor.image_std}")

```

▼ 실행 결과

```

size: SizeDict(height=224, width=224, longest_edge=None, shortest_edge=None, max_height=None, max_width=None)
mean: (0.5, 0.5, 0.5)
std: (0.5, 0.5, 0.5)

```

- 이미지 프로세서 클래스(AutoImageProcessor)는 사전 학습된 ViT 모델을 활용해 전처리 진행. (crop, resize 등의 전처리 수행 가능)
- `google/vit-base-patch16-224-in21k` 모델은 ImageNet-21k 데이터셋을 사용해 224x224 이미지 크기와 16개의 이미지 패치를 기반으로 학습됨
- 모델 학습 시 PIL.Image 형식이 아닌 Tensor 형식 사용하므로 변환 모듈 통해 이미지 변환
- ViT 데이터로더 적용

```

from torch.utils.data import DataLoader

def collator(data, transform):
    images, labels=zip(*data)
    pixel_values=torch.stack([transform(image) for image in images])
    labels=torch.tensor([label for label in labels])
    return {"pixel_values": pixel_values, "labels": labels}

train_dataloader=DataLoader(
    subset_train_dataset,
    batch_size=32,

```

```

        shuffle=True,
        collate_fn=lambda x: collator(x, transform),
        drop_last=True
    )
    valid_dataloader=DataLoader(
        subset_test_dataset,
        batch_size=4,
        shuffle=True,
        collate_fn=lambda x: collator(x, transform),
        drop_last=True
    )

    batch=next(iter(train_dataloader))
    for key, value in batch.items():
        print(f"{key}: {value.shape}")

```

▼ 실행 결과

```

pixel_values: torch.Size([32, 3, 224, 224])
labels: torch.Size([32])

```

- 데이터로더에 집합 함수(collate_fn)를 적용하기 위해 사용자 정의 함수(collator) 생성
- collator 함수에 transform 변수를 함께 전달해 이미지를 변환하고 {"pixel_values": pixel_values, "labels": labels} 구조로 값을 반환함
 - `pixel_values` : (배치 크기, 채널 수, 이미지 높이, 이미지 너비)로 구성
 - `labels` : 클래스 색인 값을 반환
- 딕셔너리 형식으로 데이터 반환하며, 배치 크기와 동일한 구조로 데이터 생성됨
- 사전 학습된 ViT 모델 불러오기

```

from transformers import ViTForImageClassification

model=ViTForImageClassification.from_pretrained(
    pretrained_model_name_or_path="google/vit-base-patch16-224-in21k",
    num_labels=len(classes),
    id2label={idx: label for label, idx in class_to_idx.items()},
    label2id=class_to_idx,
    ignore_mismatched_sizes=True
)

print(model.classifier)

```

▼ 실행 결과

- 패치 임베딩 구조 확인

▼ 실행 결과

- 패치 임베딩이 생성된 후 임베딩 함수(`model.vit.embeddings`)가 수행됨. 이 함수 앞에 [4, 1, 768] 크기의 [CLS] 토큰이 패치 앞에 붙어 197개의 벡터가 생성됨

- 하이퍼파라미터 설정

10. 비전 트랜스포머

```

eval_strategy="epoch",
learning_rate=1e-5, #초기 학습률
per_device_train_batch_size=16, #학습 배치 크기
per_device_eval_batch_size=16, #평가 배치 크기
num_train_epochs=3, #학습 반복 수
weight_decay=0.001, #가중치 감쇠
load_best_model_at_end=True, #모델 불러오기 시 최상의 모델 선택 여부
metric_for_best_model="f1", #평가방식
logging_dir="logs", #로그 저장 폴더
logging_steps=125, #로그 출력 간격
remove_unused_columns=False,
seed=7 #무작위 시드
)

```

- 매크로 평균 F1 점수

```

#매크로 평균 F1 점수
import evaluate
import numpy as np

def compute_metrics(eval_pred):
    metric=evaluate.load("f1")
    predictions, labels=eval_pred
    predictions=np.argmax(predictions, axis=1)
    macro_f1=metric.compute(
        predictions=predictions, references=labels, average="macro"
    )
    return macro_f1

```

- ViT 모델 학습

```

import torch
import evaluate
import numpy as np
from itertools import chain
from collections import defaultdict
from torch.utils.data import Subset
from torchvision import datasets
from torchvision import transforms
from transformers import AutoImageProcessor
from transformers import ViTForImageClassification
from transformers import TrainingArguments, Trainer

def subset_sampler(dataset, classes, max_len):
    target_idx=defaultdict(list)
    for idx, label in enumerate(dataset.train_labels):
        target_idx[int(label)].append(idx)

```

```

indices=list(
    chain.from_iterable(
        [target_idx[idx][:max_len] for idx in range(len(classes))]
    )
)
return Subset(dataset, indices)

def model_init(classes, class_to_idx):
    model=ViTForImageClassification.from_pretrained(
        pretrained_model_name_or_path="google/vit-base-patch16-224-in21
k",
        num_labels=len(classes),
        id2label={idx: label for label, idx in class_to_idx.items()},
        label2id=class_to_idx,
    )
    return model

def collator(data, transform):
    images, labels=zip(*data)
    pixel_values=torch.stack([transform(image) for image in images]).co
ntiguous()
    labels=torch.tensor([label for label in labels])
    return {"pixel_values": pixel_values, "labels": labels}

def compute_metrics(eval_pred):
    metric=evaluate.load("f1")
    predictions, labels=eval_pred
    predictions=np.argmax(predictions, axis=1)
    macro_f1=metric.compute(
        predictions=predictions, references=labels, average="macro"
    )
    return macro_f1

train_dataset=datasets.FashionMNIST("./datasets", download=True, train=
True)
test_dataset=datasets.FashionMNIST("./datasets", download=True, train=F
alse)

classes=train_dataset.classes
class_to_idx=train_dataset.class_to_idx

subset_train_dataset=subset_sampler(
    dataset=train_dataset, classes=train_dataset.classes, max_len=1000
)
subset_test_dataset=subset_sampler(
    dataset=test_dataset, classes=test_dataset.classes, max_len=100
)

```

```

image_processor=AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="google/vit-base-patch16-224-in21k"
)

transform=transforms.Compose(
    [
        transforms.Resize(
            size=(
                image_processor.size["height"],
                image_processor.size["width"]
            )
        ),
        transforms.Grayscale(num_output_channels=3),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
    ]
)

args=TrainingArguments(
    output_dir="./models/ViT-FashionMNIST",
    save_strategy="epoch",
    eval_strategy="epoch",
    learning_rate=1e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.001,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    logging_dir="logs",
    logging_steps=125,
    remove_unused_columns=False,
    seed=7,
    use_cpu=True,
    report_to="none",
)

trainer=Trainer(
    model_init=lambda x: model_init(classes, class_to_idx),
    args=args,
    train_dataset=subset_train_dataset,
    eval_dataset=subset_test_dataset,
    data_collator=lambda x: collator(x, transform),
    compute_metrics=compute_metrics,
    processing_class=image_processor,
)

```

```
)
trainer.train()
```

▼ 실행 결과

```
/opt/homebrew/Caskroom/miniforge/base/envs/dl-euron/lib/python3.10/site-packages/torchvision/datasets/mnist.py:66: UserWarning: train_labels has been renamed target
warnings.warn("train_labels has been renamed targets")
Fast image processor class <class 'transformers.models.vit.image_processing_vit_fast.VitImageProcessorFast'> is available for this model. Using slow image processor
Some weights of ViTForImageClassification were not initialized from the model checkpoint at google/vit-base-patch16-224-in21k and are newly initialized: ['classify']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Some weights of ViTForImageClassification were not initialized from the model checkpoint at google/vit-base-patch16-224-in21k and are newly initialized: ['classify']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

100% 1875/1875 [1:50:49<00:00, 3.43s/it]

{'loss': 1.9243, 'grad_norm': 2.6550557613372803, 'learning_rate': 9.333333333333334e-06, 'epoch': 0.2}
{'loss': 1.2731, 'grad_norm': 3.547715663909912, 'learning_rate': 8.666666666666667e-06, 'epoch': 0.4}
{'loss': 0.9353, 'grad_norm': 1.7915347814559937, 'learning_rate': 8.000000000000001e-06, 'epoch': 0.6}
{'loss': 0.7993, 'grad_norm': 2.0826196670532227, 'learning_rate': 7.333333333333333e-06, 'epoch': 0.8}
{'loss': 0.7006, 'grad_norm': 5.653544902801514, 'learning_rate': 6.666666666666667e-06, 'epoch': 1.0}

Downloading builder script: 6.79k/? [00:00<00:00, 1.43MB/s]

{'eval_loss': 0.6332910060882568, 'eval_f1': 0.8821789704543616, 'eval_runtime': 70.8088, 'eval_samples_per_second': 14.123, 'eval_steps_per_second': 0.89, 'epoch': 1.2}
{'loss': 0.6188, 'grad_norm': 2.8613698402513428, 'learning_rate': 6e-06, 'epoch': 1.2}
{'loss': 0.5693, 'grad_norm': 2.9186802940368652, 'learning_rate': 5.333333333333334e-06, 'epoch': 1.4}
{'loss': 0.5341, 'grad_norm': 2.866645574569702, 'learning_rate': 4.666666666666667e-06, 'epoch': 1.6}
{'loss': 0.5036, 'grad_norm': 2.63881516456604, 'learning_rate': 4.000000000000001e-06, 'epoch': 1.8}
{'loss': 0.477, 'grad_norm': 2.631678581237793, 'learning_rate': 3.333333333333333e-06, 'epoch': 2.0}

{'eval_loss': 0.4741559326648712, 'eval_f1': 0.9104767477860621, 'eval_runtime': 82.1952, 'eval_samples_per_second': 12.166, 'eval_steps_per_second': 0.766, 'epoch': 2.2}
{'loss': 0.4594, 'grad_norm': 3.0863828659057617, 'learning_rate': 2.666666666666667e-06, 'epoch': 2.2}
{'loss': 0.427, 'grad_norm': 6.5022053718566895, 'learning_rate': 2.0000000000000003e-06, 'epoch': 2.4}
{'loss': 0.4132, 'grad_norm': 2.04719614982605, 'learning_rate': 1.3333333333333334e-06, 'epoch': 2.6}
{'loss': 0.4165, 'grad_norm': 4.179364204406738, 'learning_rate': 6.666666666666667e-07, 'epoch': 2.8}
{'loss': 0.4171, 'grad_norm': 4.036330699920654, 'learning_rate': 0.0, 'epoch': 3.0}

{'eval_loss': 0.42744922637939453, 'eval_f1': 0.9222426363470243, 'eval_runtime': 77.7413, 'eval_samples_per_second': 12.863, 'eval_steps_per_second': 0.81, 'epoch': 3.0}
{'train_runtime': 6649.6092, 'train_samples_per_second': 4.512, 'train_steps_per_second': 0.282, 'train_loss': 0.6978945638020834, 'epoch': 3.0}

TrainOutput(global_step=1875, training_loss=0.6978945638020834, metrics={'train_runtime': 6649.6092, 'train_samples_per_second': 4.512, 'train_steps_per_second': 0.
```

• ViT 모델 성능 평가

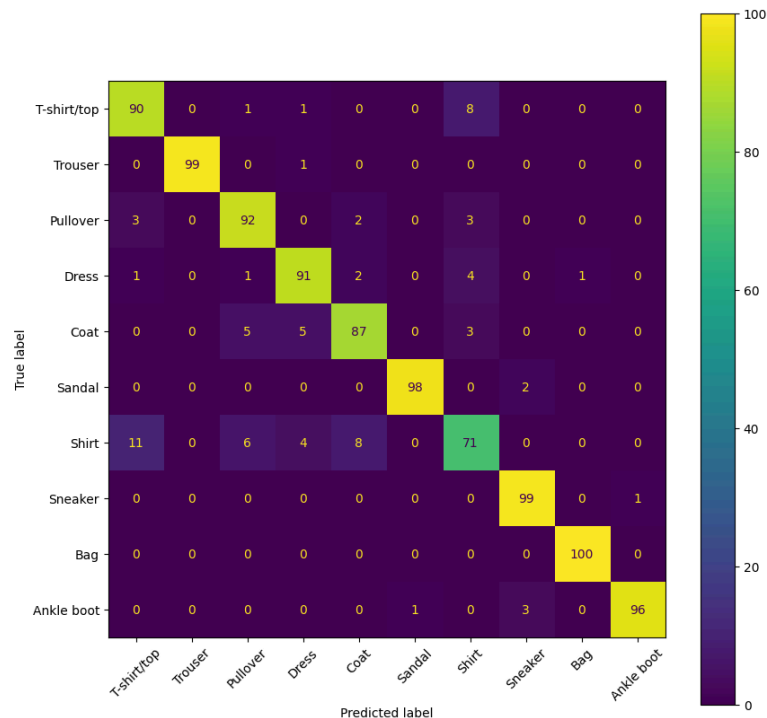
```
#모델 성능 평가
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

outputs=trainer.predict(subset_test_dataset)
print(outputs)

y_true=outputs.label_ids
y_pred=outputs.predictions.argmax(1)

labels=list(classes)
matrix=confusion_matrix(y_true, y_pred)
display=ConfusionMatrixDisplay(confusion_matrix=matrix, display_labels=
labels)
_,ax=plt.subplots(figsize=(10,10))
display.plot(xticks_rotation=45, ax=ax)
plt.show()
```

▼ 실행 결과

[illegible]

Swin Transformer

2021에 발표한 대규모 비전 인식 모델.

ViT와 Swin Transformer 차이

- ViT의 한계
 - 획일적인 패치 크기 → 다양한 이미지 크기/종횡비 처리 어려움
 - 패치 단위 입력 → 공간 정보 손실 가능
 - 패치 수 증가 시 계산량 폭증 → 대규모 데이터에서 컴퓨팅 비용 큼
- Swin Transformer의 해결 방식
 - 로컬 윈도우(Local Window)** — 전체가 아닌 윈도우 내에서만 어텐션 계산

- 시프트 윈도우(Shifted Window) — 윈도우를 이동시켜 인접 패치 간 정보 교환
- 계층적 접근 — 계층마다 패치 크기/개수를 다양하게 적용
 - 로컬 윈도우 → 세부 정보 집중
 - 시프트 윈도우 → 전역 특징 확인
 - → 고해상도 이미지도 효율적 처리

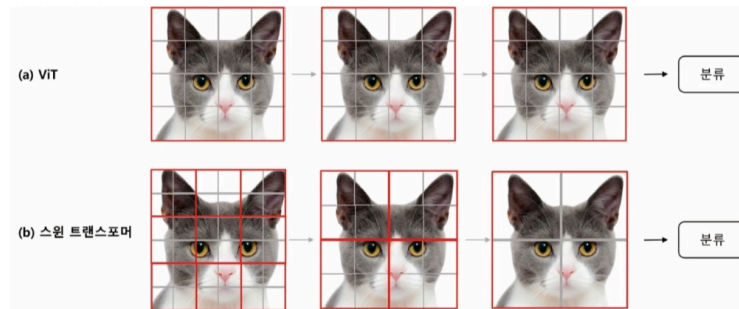


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

항목	ViT	Swin Transformer
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우	X	O
상대적 위치 편향	X	O
계층별 패치 크기	동일	다양

스윈 트랜스포머 모델 구조

- 전체 처리 흐름

패치 파티션 → 스테이지 1~4 반복 (선형 임베딩 또는 패치 병합 → 스윈 트랜스포머 블록)

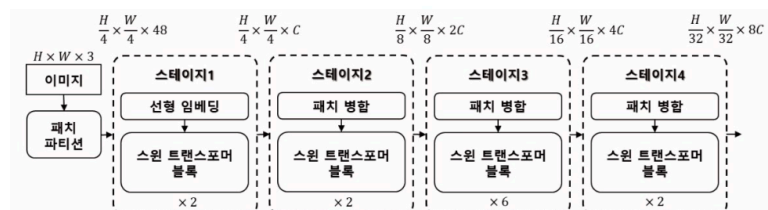


그림 10.7 스윈 트랜스포머 구조

- 4개 스테이지 구성

스테이지	첫 단계	블록 반복	해상도
1	선형 임베딩	x2	H/4 x W/4
2	패치 병합	x2	H/8 x W/8
3	패치 병합	x6	H/16 x W/16
4	패치 병합	x2	H/32 x W/32

→ 스테이지 깊어질수록 해상도↓, 채널↑ (CNN과 유사한 계층적 구조)

- 패치 병합 (Patch Merging)

- 이미지 텐서 $[C, H, W] \rightarrow [2C, H/2, W/2]$ 로 재정렬
- 매개변수 줄이면서 저차원 임베딩 수행
- 스윈 트랜스포머 블록
 - 어텐션 계층 + 계층 정규화 + 다층 퍼셉트론으로 구성
 - 두 가지 어텐션 모듈로 이루어짐
 - 윈도 격자 내부 패치 간 관계 학습
 - 윈도 격자 간 관계 학습 (시프트 윈도)

패치 파티션 / 패치 병합

- 패치 파티션 (Patch Partition) : 입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식.
 - 분할된 패치는 트랜스포머 계층의 입력으로 사용되며, 입력 이미지의 공간 정보를 보존하고 트랜스포머 계층에 대한 계산 효율성을 높임

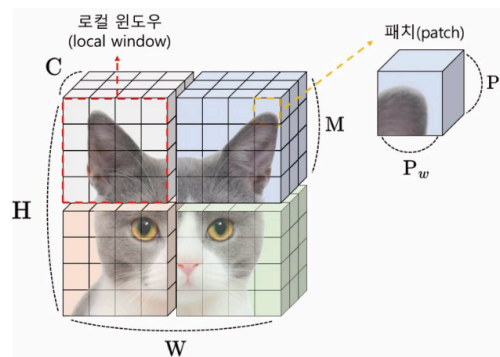


그림 10.8 패치 파티션 구조

- 일반적인 이미지 텐서는 $[C, H, W]$ 의 구조를 가짐. 이 구조에서 패치 파티션을 통해 로컬 윈도를 추가.
- 그림 10.8의 M은 로컬 윈도의 크기를 의미
- 패치 병합 (Patch Merging) : 인접한 패치들의 정보를 저차원으로 축소하는 과정.
 - 모델 매개변수의 수를 줄이기 위해 이미지 패치 텐서 $[C, H, W]$ 를 $[2C, H/2, W/2]$ 로 재정렬한 후, 2C의 차원을 저차원으로 임베딩함

스윈 트랜스포머 블록

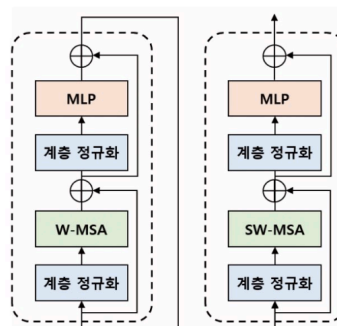


그림 10.9 스윈 트랜스포머 블록 구조

- 블록 구성
 - 계층 정규화 → **W-MSA** → 잔차 연결 → 계층 정규화 → **MLP** → 잔차 연결
 - 계층 정규화 → **SW-MSA** → 잔차 연결 → 계층 정규화 → **MLP** → 잔차 연결
- W-MSA 블록과 SW-MSA 블록이 쌍으로 순차 수행됨
- W-MSA vs. SW-MSA

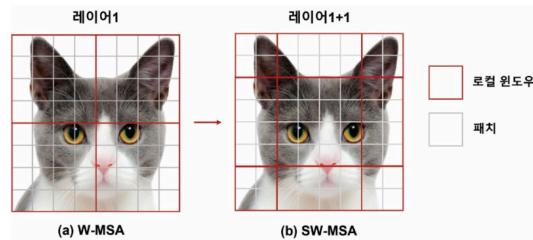


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

	W-MSA	SW-MSA
방식	로컬 윈도우 내에서만 셀프 어텐션	윈도우를 이동시켜 윈도우 간 어텐션
역할	지역 특징 추출 + 계산 복잡도 2차 → 1차 감소	윈도우 경계 너머 전역 관계 학습
단점	윈도우 내부만 봄 → 윈도우 간 정보 단절	윈도우 수 증가로 연산량 증가
해결책	효율적인 배치 계산(Batch Computation)	순환 시프트 + 어텐션 마스크

- SW-MSA 핵심 기법 : 순환 시프트 (Cyclic Shift)

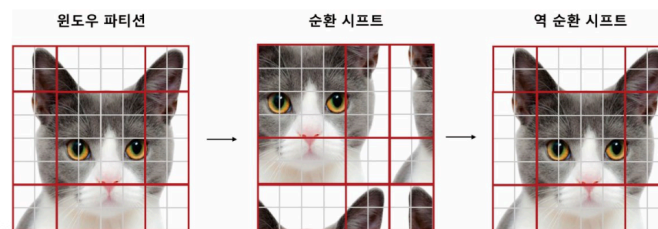


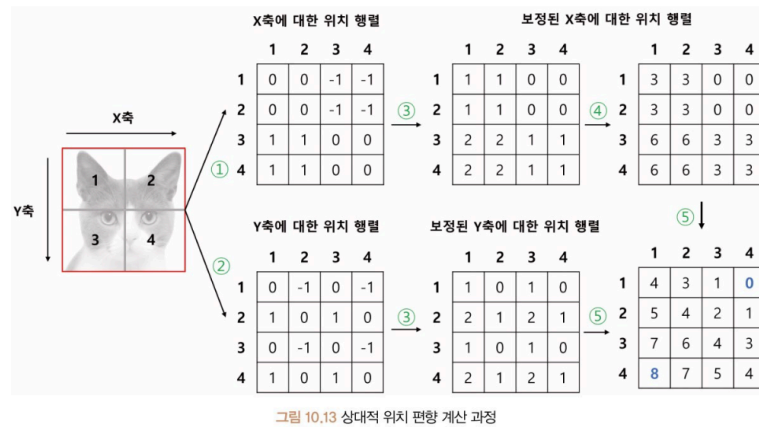
그림 10.11 순환 시프트를 활용한 SW-MSA 어텐션 계산 과정

- 로컬 윈도우를 $M/2$ 만큼 이동 → 인접 윈도우 간 정보 교환
- 이동된 위치에서 불필요한 연산은 **어텐션 마스크**로 차단
- 어텐션 계산 후 **역순환 시프트**로 원래 구조 복원
- → 효과 : 원래 9번 필요하던 셀프 어텐션을 **4번**만으로 처리 가능



그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

- 상대적 위치 편향 (Relative Position Bias)



ViT와의 차이점. 패치 간 상대적 거리를 X축(가로), Y축(세로)으로 각각 계산해 어텐션에 반영 → 위치 정보를 더 정확하게 학습

- 상대적 위치 편향 계산

```
#상대적 위치 편향 계산
import torch

window_size=2
coords_h=torch.arange(window_size)
coords_w=torch.arange(window_size)
coords=torch.stack(torch.meshgrid([coords_h, coords_w], indexing="ij"))
coords_flatten=torch.flatten(coords, 1)
relative_coords=coords_flatten[:, :, None] - coords_flatten[:, None, :]

print(relative_coords)
print(relative_coords.shape)
```

▼ 실행 결과

```
tensor([[[ 0,  0, -1, -1],
          [ 0,  0, -1, -1],
          [ 1,  1,  0,  0],
          [ 1,  1,  0,  0]],

        [[ 0, -1,  0, -1],
          [ 1,  0,  1,  0],
          [ 0, -1,  0, -1],
          [ 1,  0,  1,  0]]])
torch.Size([2, 4, 4])
```

- 위의 위치행렬 결과 이용해 상대적 위치 편향 값 산출

```
#X, Y축에 대한 위치 행렬
x_coords=relative_coords[0, :, :]
y_coords=relative_coords[1, :, :]
```

```

x_coords+=window_size-1
y_coords+=window_size-1
x_coords*=2*window_size-1
print(f"X축에 대한 행렬:\n{x_coords}\n")
print(f"Y축에 대한 행렬:\n{y_coords}\n")

relative_position_index=x_coords+y_coords
print(f"X, Y축에 대한 위치 행렬:\n{relative_position_index}")

```

▼ 실행 결과

```

X축에 대한 행렬:
tensor([[3, 3, 0, 0],
        [3, 3, 0, 0],
        [6, 6, 3, 3],
        [6, 6, 3, 3]])

Y축에 대한 행렬:
tensor([[1, 0, 1, 0],
        [2, 1, 2, 1],
        [1, 0, 1, 0],
        [2, 1, 2, 1]])

X, Y축에 대한 위치 행렬:
tensor([[4, 3, 1, 0],
        [5, 4, 2, 1],
        [7, 6, 4, 3],
        [8, 7, 5, 4]])

```

- X, Y축에 대한 상대적 위치 좌표를 벡터로 변환

```

#X, Y축에 대한 상대적 위치 좌표 변환
num_heads=1
relative_position_bias_table=torch.Tensor(
    torch.zeros((2*window_size-1)*(2*window_size-1), num_heads)
)

relative_position_bias=relative_position_bias_table[relative_position_index.view(-1)]
relative_position_bias=relative_position_bias.view(
    window_size*window_size, window_size*window_size, -1
)
print(relative_position_bias.shape)

```

▼ 실행 결과

```
torch.Size([4, 4, 1])
```

- 상대적 위치 편향을 고려한 셀프 어텐션

$$Attention(Q, K, V) = SoftMax(QK^T/\sqrt{d} + B)V$$

모델 실습

- 사전 학습된 스윈 트랜스포머 모델

```
#사전 학습된 스윈 트랜스포머 모델
from transformers import SwinForImageClassification

model=SwinForImageClassification.from_pretrained(
    pretrained_model_name_or_path="microsoft/swin-tiny-patch4-window7-2
24",
    num_labels=len(train_dataset.classes),
    id2label={idx: label for label, idx in train_dataset.class_to_idx.i
tems()},
    label2id=train_dataset.class_to_idx,
    ignore_mismatched_sizes=True
)

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children
():
                if sssub_name=="projection":
                    print("| | L", sssub_name, sssub_module)
                else:
                    print("| | L", sssub_name)
```

▼ 실행 결과

```
config.json: 71.8k/? [00:00<00:00, 7.51MB/s]
model.safetensors: 100% 113M/113M [00:13<00:00, 567MB/s]

Some weights of SwinForImageClassification were not initialized from the model checkpoint at microsoft/swin-tiny-patch4-window7-24:
- classifier.bias: found shape torch.Size([1000]) in the checkpoint and torch.Size([10]) in the model instantiated
- classifier.weight: found shape torch.Size([1000, 768]) in the checkpoint and torch.Size([10, 768]) in the model instantiated
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

swin
├─ embeddings
│   └─ L patch_embeddings
│       └─ L projection Conv2d(3, 96, kernel_size=(4, 4), stride=(4, 4))
├─ L norm
├─ L dropout
├─ encoder
│   └─ L layers
│       └─ L 0
│           └─ L 1
│               └─ L 2
│                   └─ L 3
├─ layernorm
├─ pooler
└─ classifier
```

- 패치 임베딩 모듈

```
#패치 임베딩 모듈
batch=next(iter(train_dataloader))
print("이미지 차원:", batch["pixel_values"].shape)

patch_emb_output, shape=model.swin.embeddings.patch_embeddings(batch["pixel_values"])
print("모듈:", model.swin.embeddings.patch_embeddings)
print("패치 임베딩 차원:", patch_emb_output.shape)
```

▼ 실행 결과

```
이미지 차원: torch.Size([32, 3, 224, 224])
모듈: SwinPatchEmbeddings(
  (projection): Conv2d(3, 96, kernel_size=(4, 4), stride=(4, 4))
)
패치 임베딩 차원: torch.Size([32, 3136, 96])
```

- 스윈 트랜스포머 블록

```
#스윈 트랜스포머 블록
for main_name, main_module in model.swin.encoder.layers[0].named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| L", ssub_name)
```

▼ 실행 결과

```
blocks
L 0
| L layernorm_before
| L attention
| L drop_path
| L layernorm_after
| L intermediate
| L output
L 1
| L layernorm_before
| L attention
| L drop_path
| L layernorm_after
| L intermediate
| L output
downsample
L reduction
L norm
```

- SwinLayer 구조

```
#SwinLayer 구조
print(model.swin.encoder.layers[0].blocks[0])
```

▼ 실행 결과

```
SwinLayer(
  (layernorm_before): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (attention): SwinAttention(
    (self): SwinSelfAttention(
      (query): Linear(in_features=96, out_features=96, bias=True)
      (key): Linear(in_features=96, out_features=96, bias=True)
      (value): Linear(in_features=96, out_features=96, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
    (output): SwinSelfOutput(
      (dense): Linear(in_features=96, out_features=96, bias=True)
      (dropout): Dropout(p=0.0, inplace=False)
    )
  )
  (drop_path): SwinDropPath(p=0.1)
  (layernorm_after): LayerNorm((96,), eps=1e-05, elementwise_affine=True)
  (intermediate): SwinIntermediate(
    (dense): Linear(in_features=96, out_features=384, bias=True)
    (intermediate_act_fn): GELUActivation()
  )
  (output): SwinOutput(
    (dense): Linear(in_features=384, out_features=96, bias=True)
    (dropout): Dropout(p=0.0, inplace=False)
  )
)
```

- W-MSA, SW-MSA 모듈

```
#W-MSA, SW-MSA 모듈
print("패치 임베딩 차원:", patch_emb_output.shape)

W_MSA=model.swin.encoder.layers[0].blocks[0]
SW_MSA=model.swin.encoder.layers[0].blocks[1]

W_MSA_output=W_MSA(patch_emb_output, W_MSA.input_resolution)[0]
SW_MSA_output=SW_MSA(W_MSA_output, SW_MSA.input_resolution)[0]

print("W-MSA 결과 차원:", W_MSA_output.shape)
print("SW-MSA 결과 차원:", SW_MSA_output.shape)
```

▼ 실행 결과

```
패치 임베딩 차원: torch.Size([32, 3136, 96])
W-MSA 결과 차원: torch.Size([32, 3136, 96])
SW-MSA 결과 차원: torch.Size([32, 3136, 96])
```

- 패치 병합

```
#패치 병합
patch_merge=model.swin.encoder.layers[0].downsample
print("patch_merge 모듈: ", patch_merge)

output=patch_merge(SW_MSA_output, patch_merge.input_resolution)
print("patch_merge 결과 차원: ", output.shape)
```

▼ 실행 결과

```
patch_merge 모듈: SwinPatchMerging(
  (reduction): Linear(in_features=384, out_features=192, bias=False)
  (norm): LayerNorm((384,), eps=1e-05, elementwise_affine=True)
)
patch_merge 결과 차원: torch.Size([32, 784, 192])
```

- 스윈 트랜스포머 모델 학습

```
#스윈 트랜스포머 모델 학습
from transformers import SwinForImageClassification

def subset_sampler(dataset, classes, max_len):
    target_idx=defaultdict(list)
    for idx, label in enumerate(dataset.train_labels):
        target_idx[int(label)].append(idx)

    indices=list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(dataset, indices)

def model_init(classes, class_to_idx):
    model=SwinForImageClassification.from_pretrained(
        pretrained_model_name_or_path="microsoft/swin-tiny-patch4-windo
w7-224",
        num_labels=len(classes),
        id2label={idx: label for label, idx in class_to_idx.items()},
        label2id=class_to_idx,
        ignore_mismatched_sizes=True
    )
    return model

def collator(data, transform):
    images, labels=zip(*data)
    pixel_values=torch.stack([transform(image) for image in images]).co
ntiguous()
```

```

labels=torch.tensor([label for label in labels])
return {"pixel_values": pixel_values, "labels": labels}

def compute_metrics(eval_pred):
    metric=evaluate.load("f1")
    predictions, labels=eval_pred
    predictions=np.argmax(predictions, axis=1)
    macro_f1=metric.compute(
        predictions=predictions, references=labels, average="macro"
    )
    return macro_f1

train_dataset=datasets.FashionMNIST("./datasets",download=True,train=True)
test_dataset=datasets.FashionMNIST("./datasets",download=True,train=False)

classes=train_dataset.classes
class_to_idx=train_dataset.class_to_idx

subset_train_dataset=subset_sampler(
    dataset=train_dataset, classes=train_dataset.classes, max_len=1000
)
subset_test_dataset=subset_sampler(
    dataset=test_dataset, classes=test_dataset.classes, max_len=100
)

image_processor=AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="microsoft/swin-tiny-patch4-window7-224"
)

transform=transforms.Compose(
    [
        transforms.Resize(
            size=(
                image_processor.size["height"],
                image_processor.size["width"]
            )
        ),
        transforms.Grayscale(num_output_channels=3),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
    ]
)

```

```

args=TrainingArguments(
    output_dir="./models/Swin-FashionMNIST",
    save_strategy="epoch",
    eval_strategy="epoch",
    learning_rate=1e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.001,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    logging_dir="logs",
    logging_steps=125,
    remove_unused_columns=False,
    seed=7,
    use_cpu=True
)

trainer=Trainer(
    model_init=lambda x: model_init(classes, class_to_idx),
    args=args,
    train_dataset=subset_train_dataset,
    eval_dataset=subset_test_dataset,
    data_collator=lambda x: collator(x, transform),
    compute_metrics=compute_metrics,
    tokenizer=image_processor,
)

trainer.train()

```

▼ 실행 결과

```

/var/folders/4g/3301trc50fz6_vl0ox1b71r0000gn/T/ipykernel_29921/370442472.py:92: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0 for
trainer=Trainer(
Some weights of SwinForImageClassification were not initialized from the model checkpoint at microsoft/swin-tiny-patch4-window7-224 and are newly initialized because
- classifier.bias: found shape torch.Size([1000]) in the checkpoint and torch.Size([10]) in the model instantiated
- classifier.weight: found shape torch.Size([1000, 768]) in the checkpoint and torch.Size([10, 768]) in the model instantiated
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Some weights of SwinForImageClassification were not initialized from the model checkpoint at microsoft/swin-tiny-patch4-window7-224 and are newly initialized because
- classifier.bias: found shape torch.Size([1000]) in the checkpoint and torch.Size([10]) in the model instantiated
- classifier.weight: found shape torch.Size([1000, 768]) in the checkpoint and torch.Size([10, 768]) in the model instantiated
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

100% 1875/1875 [1:01:59<00:00, 1.89s/it]

{'loss': 1.2381, 'grad_norm': 72.6116714477539, 'learning_rate': 9.333333333333334e-06, 'epoch': 0.2}
{'loss': 0.5515, 'grad_norm': 28.99755096435547, 'learning_rate': 8.666666666666667e-06, 'epoch': 0.4}
{'loss': 0.4167, 'grad_norm': 21.601491928100586, 'learning_rate': 8.000000000000001e-06, 'epoch': 0.6}
{'loss': 0.4057, 'grad_norm': 39.313941955566406, 'learning_rate': 7.333333333333333e-06, 'epoch': 0.8}
{'loss': 0.3699, 'grad_norm': 52.97936248779297, 'learning_rate': 6.666666666666667e-06, 'epoch': 1.0}

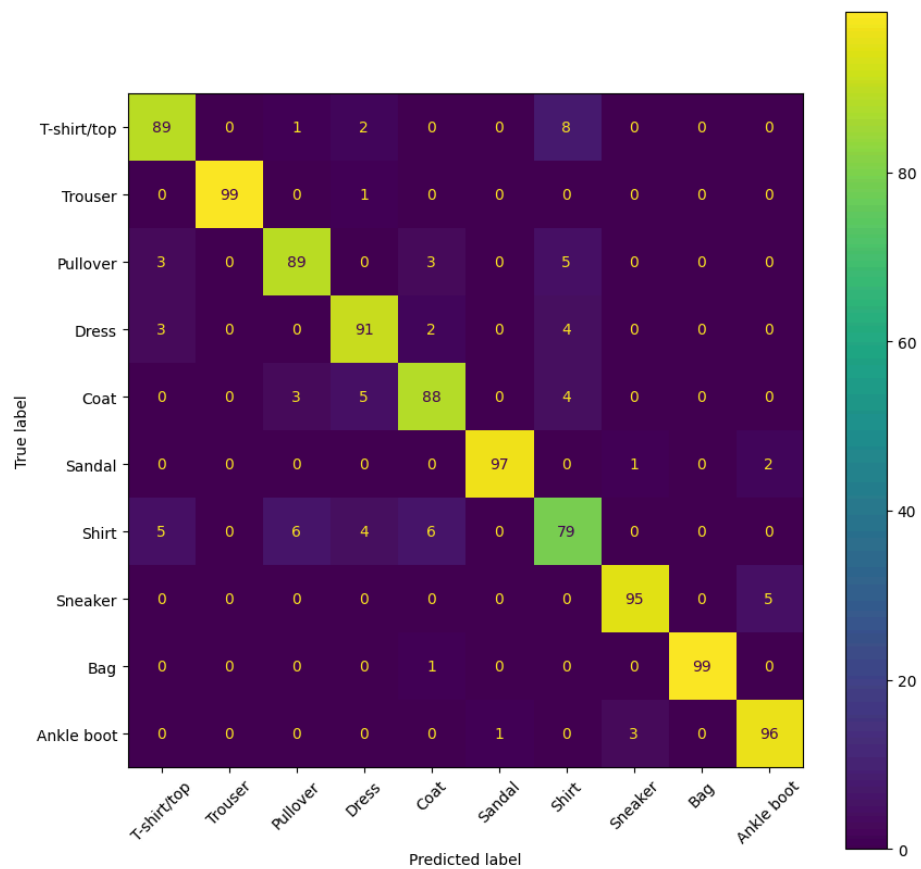
{'eval_loss': 0.301453560590744, 'eval_f1': 0.8993184151971685, 'eval_runtime': 52.9204, 'eval_samples_per_second': 18.896, 'eval_steps_per_second': 1.19, 'epoch':
{'loss': 0.3343, 'grad_norm': 55.2942008972168, 'learning_rate': 6e-06, 'epoch': 1.2}
{'loss': 0.3082, 'grad_norm': 22.593252182006836, 'learning_rate': 5.333333333333334e-06, 'epoch': 1.4}
{'loss': 0.3089, 'grad_norm': 4.474065780639648, 'learning_rate': 4.666666666666667e-06, 'epoch': 1.6}
{'loss': 0.2812, 'grad_norm': 5.404590129852295, 'learning_rate': 4.000000000000001e-06, 'epoch': 1.8}
{'loss': 0.2669, 'grad_norm': 9.930685997009277, 'learning_rate': 3.333333333333333e-06, 'epoch': 2.0}

{'eval_loss': 0.2588740289211273, 'eval_f1': 0.911599208620891, 'eval_runtime': 52.852, 'eval_samples_per_second': 18.921, 'eval_steps_per_second': 1.192, 'epoch':
{'loss': 0.2702, 'grad_norm': 17.918113708496094, 'learning_rate': 2.666666666666667e-06, 'epoch': 2.2}
{'loss': 0.2294, 'grad_norm': 21.30327796936035, 'learning_rate': 2.0000000000000003e-06, 'epoch': 2.4}
{'loss': 0.2343, 'grad_norm': 14.836939811706543, 'learning_rate': 1.3333333333333334e-06, 'epoch': 2.6}
{'loss': 0.2559, 'grad_norm': 27.545644760131836, 'learning_rate': 6.666666666666667e-07, 'epoch': 2.8}
{'loss': 0.247, 'grad_norm': 22.445945739746094, 'learning_rate': 0.0, 'epoch': 3.0}

{'eval_loss': 0.24512597918510437, 'eval_f1': 0.9221358491717183, 'eval_runtime': 49.3542, 'eval_samples_per_second': 20.262, 'eval_steps_per_second': 1.276, 'epoch':
{'train_runtime': 3719.3252, 'train_samples_per_second': 8.066, 'train_steps_per_second': 0.504, 'train_loss': 0.3812076538085937, 'epoch': 3.0}

TrainOutput(global_step=1875, training_loss=0.3812076538085937, metrics={'train_runtime': 3719.3252, 'train_samples_per_second': 8.066, 'train_steps_per_second': 0.

```

CvT(Convolutional Vision Transformer)

합성곱 신경망 구조에서 활용하는 계층형 구조(Hierarchical Architecture)를 ViT에 적용한 모델

- 등장 배경
 - ViT는 모든 패치에 동일한 크기/간격 적용 → 물체 크기나 해상도를 계층적으로 학습하기 어려움
 - Swin은 로컬 윈도우 크기를 다르게 해서 해결했다면, CvT는 합성곱 계층형 구조 자체를 ViT에 접목
- CvT의 핵심 아이디어
 - 트랜스포머 입력 전에 합성곱 연산으로 특징 맵 먼저 추출
 - 추출된 특징을 겹치는 패치로 나눠 트랜스포머에 입력
 - 지역 특징(CNN) + 전역 특징(Self-Attention) 동시 활용
- 계층별 학습 내용

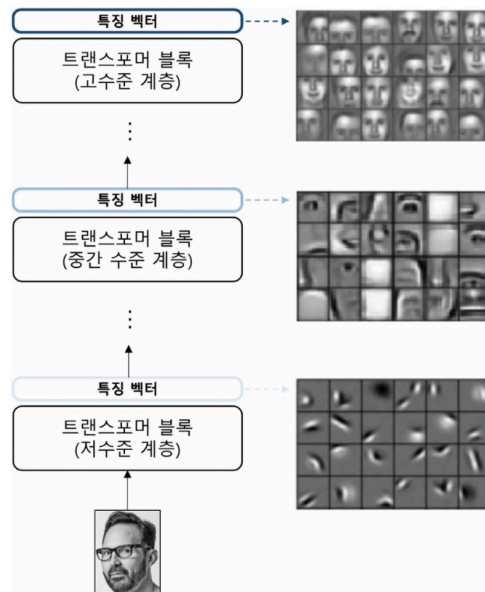


그림 10.14 CvT 모델의 계층적 특징 구조 예시

계층	학습하는 특징
저수준 (Low-level)	선, 점 등 단순 패턴
중간 수준 (Mid-level)	눈, 귀 등 부분 구조
고수준 (High-level)	얼굴 전체 구조 등 추상적 특징

→ 물체의 크기와 해상도를 더 잘 표현

• ViT vs. CvT 비교

항목	ViT	CvT
위치 임베딩	O	X (불필요)
패치 임베딩	겹치지 않는 선형 임베딩	겹치는 합성곱 임베딩
어텐션 임베딩	선형 임베딩	합성곱 임베딩
계층형 구조	X	O

→ CvT는 위치 임베딩 없이도 다양한 해상도 처리 가능, Swin보다 적은 매개변수로 유사한 성능

합성곱 토큰 임베딩

기존 ViT는 패치를 선형 임베딩으로 Q/K/V 변환 → 이미지의 2D 공간 구조 손실

CvT는 2D 합성곱 임베딩으로 Q/K/V 변환 → 2D 구조 보존하면서 셀프 어텐션 계산

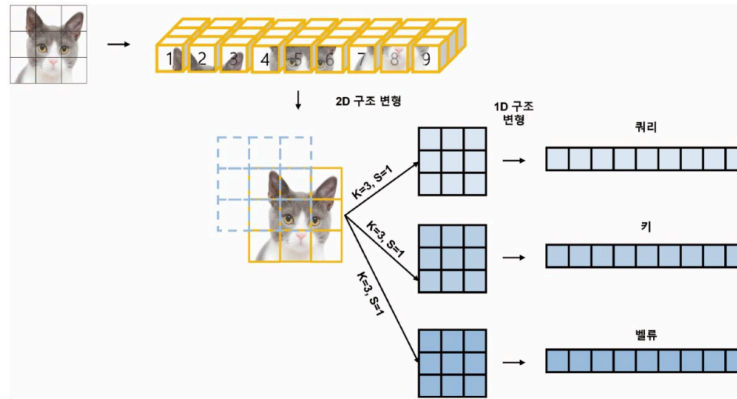


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

- 처리 흐름
 1. 이미지를 9개 패치로 분할 → 1D 시퀀스
 2. 2D 구조로 재변형 (토큰 맵 복원)
 3. 2D 합성곱 연산 ($K=3, S=1$) 을 Q, K, V 각각에 독립적으로 적용
 4. 결과를 다시 1D 구조로 변형 → 기존 셀프 어텐션 방식으로 학습
- 합성곱 임베딩의 효과

효과	내용
지역 특징 학습	합성곱으로 인접 패치 간 관계 포착
시퀀스 길이 감소	다운 샘플링 효과
특징 맵 채널 증가	CNN처럼 깊어질수록 표현력 향상
2D 구조 보존	위치 임베딩 없이도 공간 정보 유지

어텐션에 대한 합성곱 임베딩

Q/K/V를 동일한 차원으로 쓰는 대신, K와 V의 stride를 키워 패치 수를 줄임 → 연산량 감소

- Q/K/V 설정 비교

	커널(K)	간격(S)	패치 수
Query	3×3	1	9개
Key	3×3	2	4개
Value	3×3	2	4개

→ $Q(9) \times K(4) =$ 어텐션 맵 9×4

→ 어텐션 맵(9×4) $\times$ V(4) = 셀프 어텐션 벡터 9×1 (출력 크기는 입력과 동일하게 유지)

- 계층이 깊어질수록
 - 패치 크기↓ + 임베딩 차원↑ (패치 크기 4배 감소 → 차원 4배 증가)
 - e.g. 64×64 패치, 128차원 → 8×8 패치, 512차원
 - 패치 수가 줄어든 만큼 차원을 늘려 정보 손실 없이 매개변수 수 대폭 감소, 네트워크 안정성 향상

모델 실습

- CvT 모델 이미지 데이터 전처리

```
from transformers import AutoImageProcessor

image_processor=AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="microsoft/cvt-21"
)

transform=transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Resize(
            size=(
                image_processor.size["shortest_edge"],
                image_processor.size["shortest_edge"]
            )
        ),
        transforms.Lambda(
            lambda x: torch.cat([x,x,x], 0)
        ),
        transforms.Normalize( #정규화
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
    ]
)

print(f"size: {image_processor.size}")
print(f"mean: {image_processor.image_mean}")
print(f"std: {image_processor.image_std}")
```

▼ 실행 결과

```
preprocessor_config.json: 100% ██████████ 266/266 [00:00<00:00, 58.7kB/s]

size: {'shortest_edge': 224}
mean: [0.485, 0.456, 0.406]
std: [0.229, 0.224, 0.225]
```

- 사전 학습된 CvT 모델

```
#사전 학습된 CvT 모델
from transformers import CvtForImageClassification

model=CvtForImageClassification.from_pretrained(
    pretrained_model_name_or_path="microsoft/cvt-21",
    num_labels=len(train_dataset.classes),
    id2label={idx: label for label, idx in train_dataset.class_to_idx.items()},
    label2id=train_dataset.class_to_idx,
```

```

        ignore_mismatched_sizes=True
    )

    for main_name, main_module in model.named_children():
        print(main_name)
        for sub_name, sub_module in main_module.named_children():
            print("L", sub_name)
            for ssub_name, ssub_module in sub_module.named_children():
                print(" L", ssub_name)
                for sssub_name, sssub_module in ssub_module.named_children
                    print("  L", sssub_name)

```

▼ 실행 결과

```

config.json: 70.3k/? [00:00<00:00, 9.39MB/s]
pytorch_model.bin: 100% 127M/127M [00:46<00:00, 634MB/s]

Some weights of CvtfForImageClassification were not initialized from the model checkpoint at microsoft/cvt-21 and are newly initialized because the shapes did not ma
- classifier.weight: found shape torch.Size([1000, 384]) in the checkpoint and torch.Size([10, 384]) in the model instantiated
- classifier.bias: found shape torch.Size([1000]) in the checkpoint and torch.Size([10]) in the model instantiated
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

cvt
├── encoder
│   ├── stages
│   │   ├── 0
│   │   ├── 1
│   │   └── 2
│   └── layernorm
└── classifier
model.safetensors: 100% 127M/127M [00:50<00:00, 14.5MB/s]

```

• CvT 모델의 스테이지 구조

```

#CvT 모델의 스테이지 구조
stages=model.cvt.encoder.stages
print(stages[0])

```

▼ 실행 결과

```

CvtStage(
  (embedding): CvtfEmbeddings(
    (convolution_embeddings): CvtfConvEmbeddings(
      (projection): Conv2d(3, 64, kernel_size=(7, 7), stride=(4, 4), padding=(2, 2))
      (normalization): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
    )
    (dropout): Dropout(p=0.0, inplace=False)
  )
  (layers): Sequential(
    (0): CvtLayer(
      (attention): CvtfAttention(
        (attention): CvtfSelfAttention(
          (convolution_projection_query): CvtfSelfAttentionProjection(
            (convolution_projection): CvtfSelfAttentionConvProjection(
              (convolution): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=64, bias=False)
              (normalization): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            )
            (linear_projection): CvtfSelfAttentionLinearProjection()
          )
          (convolution_projection_key): CvtfSelfAttentionProjection(
            (convolution_projection): CvtfSelfAttentionConvProjection(
              (convolution): Conv2d(64, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), groups=64, bias=False)
              (normalization): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
            )
            (linear_projection): CvtfSelfAttentionLinearProjection()
          )
        )
      )
      (layernorm_after): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
    )
  )
)
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

- 셀프 어텐션 적용

```
#셀프 어텐션 적용
batch=next(iter(train_dataloader))
print("이미지 차원:", batch["pixel_values"].shape)
patch_emb_output=stages[0].embedding(batch["pixel_values"])
print("패치 임베딩 차원:", patch_emb_output.shape)

batch_size, num_channels, height, width=patch_emb_output.shape
hidden_state=patch_emb_output.view(batch_size, num_channels, height*width).permute(0, 2, 1)
print("셀프 어텐션 입력 차원:", hidden_state.shape)

attention_output=stages[0].layers[0].attention.attention(hidden_state, height, width)
print("셀프 어텐션 출력 차원:", attention_output.shape)
```

▼ 실행 결과

```
이미지 차원: torch.Size([32, 3, 224, 224])
패치 임베딩 차원: torch.Size([32, 64, 56, 56])
셀프 어텐션 입력 차원: torch.Size([32, 3136, 64])
셀프 어텐션 출력 차원: torch.Size([32, 3136, 64])
```

- CvT 모델 학습

```
#모델 학습
from transformers import CvtForImageClassification

def subset_sampler(dataset, classes, max_len):
    target_idx=defaultdict(list)
    for idx, label in enumerate(dataset.train_labels):
        target_idx[int(label)].append(idx)

    indices=list(
        chain.from_iterable(
            [target_idx[idx][:max_len] for idx in range(len(classes))]
        )
    )
    return Subset(dataset, indices)

def model_init(classes, class_to_idx):
    model=CvtForImageClassification.from_pretrained(
        pretrained_model_name_or_path="microsoft/cvt-21",
        num_labels=len(classes),
        id2label={idx: label for label, idx in class_to_idx.items()},
        label2id=class_to_idx,
        ignore_mismatched_sizes=True
    )
```

```

return model

def collator(data, transform):
    images, labels=zip(*data)
    pixel_values=torch.stack([transform(image) for image in images])
    labels=torch.tensor([label for label in labels])
    return {"pixel_values": pixel_values, "labels": labels}

def compute_metrics(eval_pred):
    metric=evaluate.load("f1")
    predictions, labels=eval_pred
    predictions=np.argmax(predictions, axis=1)
    macro_f1=metric.compute(
        predictions=predictions, references=labels, average="macro"
    )
    return macro_f1

train_dataset=datasets.FashionMNIST(root="./datasets", download=True, train=True)
test_dataset=datasets.FashionMNIST(root="./datasets", download=True, train=False)

classes=train_dataset.classes
class_to_idx=train_dataset.class_to_idx

subset_train_dataset=subset_sampler(
    dataset=train_dataset, classes=train_dataset.classes, max_len=1000
)
subset_test_dataset=subset_sampler(
    dataset=test_dataset, classes=test_dataset.classes, max_len=100
)

image_processor=AutoImageProcessor.from_pretrained(
    pretrained_model_name_or_path="microsoft/cvt-21"
)
transform=transforms.Compose(
    [
        transforms.Resize(
            size=(
                image_processor.size["height"],
                image_processor.size["width"]
            )
        ),
        transforms.Grayscale(num_output_channels=3),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=image_processor.image_mean,
            std=image_processor.image_std
        )
    ]
)

```

```

    )
]
)
args=TrainingArguments(
    output_dir="./models/CvT-FashionMNIST",
    save_strategy="epoch",
    eval_strategy="epoch",
    learning_rate=1e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.001,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    logging_dir="logs",
    logging_steps=125,
    remove_unused_columns=False,
    seed=7,
    use_cpu=True
)

trainer=Trainer(
    model_init=lambda x: model_init(classes, class_to_idx),
    args=args,
    train_dataset=subset_train_dataset,
    eval_dataset=subset_test_dataset,
    data_collator=lambda x: collator(x, transform),
    compute_metrics=compute_metrics,
    tokenizer=image_processor #토큰을 임베딩해 모델 입력으로 사용하도록 토큰 분
리 작업 수행
)
trainer.train()

```

```

import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

outputs=trainer.predict(subset_test_dataset)
print(outputs)

y_true=outputs.label_ids
y_pred=outputs.predictions.argmax(1)

labels=list(classes)
matrix=confusion_matrix(y_true, y_pred)
display=ConfusionMatrixDisplay(confusion_matrix=matrix, display_labels=
labels)

```

▼ 실행 결과

	T-shirt/top	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Ankle boot
T-shirt/top	88	0	1	2	0	0	9	0	0	0
Trouser	0	99	0	1	0	0	0	0	0	0
Pullover	3	0	90	0	2	0	5	0	0	0
Dress	3	0	0	91	1	0	5	0	0	0
Coat	0	0	4	6	86	0	4	0	0	0
Sandal	0	0	0	0	0	97	0	1	0	2
Shirt	5	0	6	4	7	0	78	0	0	0
Sneaker	0	0	0	0	0	0	0	95	0	5
Bag	0	0	0	0	1	0	0	0	99	0
Ankle boot	0	0	0	0	0	1	0	3	0	96